```
import pandas as pd
import numpy as np
import ast

# 1. Load the raw files
df_games = pd.read_csv('games.csv')
df_vgsales = pd.read_csv('vgsales.csv')

print(f"Initial games shape: {df_games.shape}")
print(f"Initial vgsales shape: {df_vgsales.shape}")

# Inspect column types and initial missing values
print("\n--- Missing values in games.csv ---")
print(df_games.isnull().sum()[df_games.isnull().sum() > 0])

print("\n--- Missing values in vgsales.csv ---")
print(df_vgsales.isnull().sum()[df_vgsales.isnull().sum() > 0])
```

```
Initial games shape: (1512, 14)
Initial vgsales shape: (16598, 11)

--- Missing values in games.csv ---
Team        1
Rating     13
Summary     1
dtype: int64

--- Missing values in vgsales.csv ---
Year         271
Publisher     58
dtype: int64
```

```
import pandas as pd
import numpy as np
import ast

# 1. Load raw data
df_games = pd.read_csv('games.csv')
df_vgsales = pd.read_csv('vgsales.csv')

# ==========================================
# A. CLEAN games.csv
# ==========================================
# Drop unnecessary index column and text columns causing Workbench
parser crash
cols_to_drop = [c for c in ['Unnamed: 0', 'Summary', 'Reviews'] if c
in df_games.columns]
df_games = df_games.drop(columns=cols_to_drop)

# Convert 'K' string representations into integers
def parse_metric(val):
```

```
    if pd.isna(val):
        return 0
    val_str = str(val).strip().upper()
    if 'K' in val_str:
        return int(float(val_str.replace('K', '')) * 1000)
    try:
        return int(float(val_str))
    except ValueError:
        return 0

metric_cols = ['Plays', 'Playing', 'Backlogs', 'Wishlist', 'Times
Listed', 'Number of Reviews']
for col in metric_cols:
    if col in df_games.columns:
        df_games[col] = df_games[col].apply(parse_metric)

# Clean bracketed string lists (Genres & Team)
def clean_list_format(val):
    if pd.isna(val):
        return 'Unknown'
    try:
        parsed = ast.literal_eval(val)
        if isinstance(parsed, list):
            return ', '.join([str(item).strip() for item in parsed if
item])
    except Exception:
        pass
    return str(val).strip("[]'\" ")

df_games['Team'] =
df_games['Team'].apply(clean_list_format).replace('', 'Unknown')
df_games['Genres'] =
df_games['Genres'].apply(clean_list_format).replace('', 'Unknown')
df_games['Primary_Genre'] = df_games['Genres'].apply(lambda x:
x.split(',')[0].strip() if x else 'Unknown')

# Handle numeric ratings & dates
df_games['Rating'] =
df_games['Rating'].fillna(df_games['Rating'].median()).round(2)
df_games['Release_Date'] = pd.to_datetime(df_games['Release Date'],
errors='coerce').dt.strftime('%Y-%m-%d')
if 'Release Date' in df_games.columns:
    df_games = df_games.drop(columns=['Release Date'])

# Normalize Title and drop duplicates
df_games['Title'] =
df_games['Title'].astype(str).str.strip().str.replace('"', '',
regex=False)
df_games = df_games.drop_duplicates(subset=['Title'], keep='first')
```

```
# Rename to clean SQL column names
df_games = df_games.rename(columns={
    'Times Listed': 'Times_Listed',
    'Number of Reviews': 'Number_of_Reviews'
})


# ========================================
# B. CLEAN vgsales.csv
# ========================================
df_vgsales['Name'] =
df_vgsales['Name'].astype(str).str.strip().str.replace('"', '',
regex=False)
df_vgsales['Platform'] =
df_vgsales['Platform'].astype(str).str.strip().str.upper()
df_vgsales['Genre'] = df_vgsales['Genre'].astype(str).str.strip()
df_vgsales['Publisher'] =
df_vgsales['Publisher'].fillna('Unknown').astype(str).str.strip()

median_year = int(df_vgsales['Year'].median())
df_vgsales['Year'] =
df_vgsales['Year'].fillna(median_year).astype(int)

sales_cols = ['NA_Sales', 'EU_Sales', 'JP_Sales', 'Other_Sales',
'Global_Sales']
for col in sales_cols:
    df_vgsales[col] = pd.to_numeric(df_vgsales[col],
errors='coerce').fillna(0.0).round(2)

df_vgsales = df_vgsales.drop_duplicates(subset=['Name', 'Platform'],
keep='first')


# ========================================
# C. VERIFICATION & EXPORT
# ========================================
print("Games Nulls:\n", df_games.isnull().sum()
[df_games.isnull().sum() > 0])
print("Sales Nulls:\n", df_vgsales.isnull().sum()
[df_vgsales.isnull().sum() > 0])

# Export clean CSVs (standard utf-8)
df_games.to_csv('clean_games.csv', index=False, encoding='utf-8')
df_vgsales.to_csv('clean_vgsales.csv', index=False, encoding='utf-8')

# Merged CSV
df_merged = pd.merge(df_vgsales, df_games, left_on='Name',
right_on='Title', how='inner')
df_merged.to_csv('clean_merged_vgsales.csv', index=False,
encoding='utf-8')
```

```
print("\nDone! All files re-saved cleanly.")
```

```
Games Nulls:
 Release_Date    3
dtype: int64
Sales Nulls:
 Series([], dtype: int64)

Done! All files re-saved cleanly.
```

```sql
-- ================================================================
--   games.csv (Game Metadata Only)
-- ================================================================
-- 1. What are the top-rated games by user reviews?
     #Top-rated games by user reviews (with a minimum review threshold to avoid 1-review skew)
SELECT Title, Rating, Number_of_Reviews, Plays
FROM clean_games
WHERE Number_of_Reviews > 100
ORDER BY Rating DESC, Number_of_Reviews DESC
LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Title	Rating	Number_of_Reviews	Plays
Outer Wilds	4.6	1800	7700
Disco Elysium: The Final Cut	4.6	1100	6000
Disco Elysium	4.6	1100	4000
Metal Gear Solid 3: Subsistence	4.6	641	3700
Half-Life: Alyx	4.6	515	1900
Final Fantasy XIV: Endwalker	4.6	426	2500
Final Fantasy XIV: Shadowbringers	4.6	40	3000
The Great Ace Attorney 2: Resolve	4.6	386	1100
Umineko: When They Cry Chiru	4.6	324	1700

426

clean_games 2

Output

Action Output

#	Time	Action	Message
1	18:22:37	SELECT Title, Rating, Number_of_Reviews, Plays FROM clean_games WHERE Numbe...	10 row(s) returned
2	18:23:13	SELECT Title, Rating, Number_of_Reviews, Plays FROM clean_games WHERE Numbe...	10 row(s) returned

```sql
-- 2. Which developers (Teams) have the highest average ratings?
    #Developers (Teams) with the highest average ratings (at least 2 games)
SELECT Team, ROUND(AVG(Rating), 2) AS avg_rating, COUNT(*) AS games_released
FROM clean_games
WHERE Team <> 'Unknown'
GROUP BY Team
HAVING COUNT(*) >= 2
ORDER BY avg_rating DESC, games_released DESC
LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Team	avg_rating	games_released
ZA/UM	4.6	2
FromSoftware, Sony Computer Entertainment	4.55	2
Mobius Digital, Annapurna Interactive	4.55	2
Konami Computer Entertainment Japan, Konami	4.5	3
Team Silent, Konami	4.5	2
Team Cherry	4.45	2
Nitroplus, JAST USA	4.45	2
Square Enix, PlatinumGames		
Buena Vista Games, Square Enix	4.4	2

Square Enix, PlatinumGames

Result 3

Output

Action Output

#	Time	Action	Message
1	17:21:44	SELECT Title, Rating, Number_of_Reviews, Plays FROM clean_games WHERE Numbe...	10 row(s) returned
2	17:23:03	SELECT Team, ROUND(AVG(Rating), 2) AS avg_rating, COUNT(*) AS games_released ...	10 row(s) returned

```sql
-- 3. What are the most common genres in the dataset?
    #Most common genres in the dataset
SELECT Primary_Genre, COUNT(*) AS total_games
FROM clean_games
GROUP BY Primary_Genre
ORDER BY total_games DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Primary_Genre	total_games
Adventure	713
RPG	77
Shooter	58
Arcade	43
Indie	37
Fighting	37
Platform	34
Brawler	26
Racing	14

Result 4

Output

Action Output

#	Time	Action	Message
1	17:21:44	SELECT Title, Rating, Number_of_Reviews, Plays FROM clean_games WHERE Numbe...	10 row(s) returned
2	17:23:03	SELECT Team, ROUND(AVG(Rating), 2) AS avg_rating, COUNT(*) AS games_released ...	10 row(s) returned
3	17:24:23	SELECT Primary_Genre, COUNT(*) AS total_games FROM clean_games GROUP BY Pri...	20 row(s) returned

```sql
-- 4. Which games have the highest backlog compared to wishlist?
      #Games with highest backlog compared to wishlist (ratio)
SELECT
    Title,
    Backlogs,
    Wishlist,
    ROUND(Backlogs / NULLIF(Wishlist, 0), 2) AS backlog_to_wishlist_ratio
FROM clean_games
WHERE Wishlist > 50
ORDER BY backlog_to_wishlist_ratio DESC
LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Title	Backlogs	Wishlist	backlog_to_wishlist_ratio
Half-Life: Opposing Force	838	95	8.82
Destiny 2	845	103	8.20
BioShock 2 Remastered	2000	246	8.13
Borderlands: The Pre-Sequel	1900	238	7.98
Killing Floor 2	501	66	7.59
LEGO Batman 3: Beyond Gotham	801	113	7.09
Just Cause 3	1000	155	6.45
BioShock Remastered	2200	341	6.45
For Honor	579	91	6.36

Result 5

Output

Action Output

#	Time	Action	Message
1	17:21:44	SELECT Title, Rating, Number_of_Reviews, Plays FROM clean_games WHERE Numbe...	10 row(s) returned
2	17:23:03	SELECT Team, ROUND(AVG(Rating), 2) AS avg_rating, COUNT(*) AS games_released ...	10 row(s) returned
3	17:24:23	SELECT Primary_Genre, COUNT(*) AS total_games FROM clean_games GROUP BY Pri...	20 row(s) returned
4	17:26:12	SELECT Title, Backlogs, Wishlist, ROUND(Backlogs / NULLIF(Wishlist, 0), ...	10 row(s) returned

```sql
-- 5. What is the game release trend across years?
    #Game release trend across years
SELECT
    YEAR(Release_Date) AS release_year,
    COUNT(*) AS total_releases
FROM clean_games
WHERE Release_Date IS NOT NULL
GROUP BY release_year
ORDER BY release_year ASC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

release_year	total_releases
NULL	3
1980	1
1982	1
1985	1
1986	3
1987	3
1988	3
1989	2
1990	6

Result 6

Output

Action Output

#	Time	Action	Message
1	17:21:44	SELECT Title, Rating, Number_of_Reviews, Plays FROM clean_games WHERE Numbe...	10 row(s) returned
2	17:23:03	SELECT Team, ROUND(AVG(Rating), 2) AS avg_rating, COUNT(*) AS games_released ...	10 row(s) returned
3	17:24:23	SELECT Primary_Genre, COUNT(*) AS total_games FROM clean_games GROUP BY Pri...	20 row(s) returned
4	17:26:12	SELECT Title, Backlogs, Wishlist, ROUND(Backlogs / NULLIF(Wishlist, 0), ...	10 row(s) returned
5	17:27:24	SELECT YEAR(Release_Date) AS release_year, COUNT(*) AS total_releases FR...	44 row(s) returned

```sql
-- 6. What is the distribution of user ratings?
    #Distribution of user ratings (Bucket breakdown)
SELECT
    FLOOR(Rating * 2) / 2 AS rating_bin,
    COUNT(*) AS game_count
FROM clean_games
GROUP BY rating_bin
ORDER BY rating_bin DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

rating_bin	game_count
4.5	37
4	320
3.5	416
3	216
2.5	82
2	19
1.5	6
1	2
0.5	1

Result 7

Output

Action Output

#	Time	Action	Message
1	17:21:44	SELECT Title, Rating, Number_of_Reviews, Plays FROM clean_games WHERE Numbe...	10 row(s) returned
2	17:23:03	SELECT Team, ROUND(AVG(Rating), 2) AS avg_rating, COUNT(*) AS games_released ...	10 row(s) returned
3	17:24:23	SELECT Primary_Genre, COUNT(*) AS total_games FROM clean_games GROUP BY Pri...	20 row(s) returned
4	17:26:12	SELECT Title, Backlogs, Wishlist, ROUND(Backlogs / NULLIF(Wishlist, 0), ...	10 row(s) returned
5	17:27:24	SELECT YEAR(Release_Date) AS release_year, COUNT(*) AS total_releases FR...	44 row(s) returned
6	17:28:30	SELECT FLOOR(Rating * 2) / 2 AS rating_bin, COUNT(*) AS game_count FROM ...	9 row(s) returned

```sql
-- 7. What are the top 10 most wishlisted games?
    #Top 10 most wishlisted games
SELECT Title, Wishlist, Plays, Rating
FROM clean_games
ORDER BY Wishlist DESC
LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Title	Wishlist	Plays	Rating
The Legend of Zelda: Tears of the Kingdom	5400	72	3.8
Elden Ring	4800	17000	4.5
Omori	3800	7200	4.2
NieR Replicant ver.1.22474487139...	3700	6700	4.2
Hades			
NieR: Automata	3600	18000	4.3
Stray	3400	7700	3.7
Sekiro: Shadows Die Twice	3400	14000	4.4
Metroid Dread	3300	9200	4.3

NieR Replicant ver.1.22474487139...

clean_games 8

Output

Action Output

#	Time	Action	Message
2	17:23:03	SELECT Team, ROUND(AVG(Rating), 2) AS avg_rating, COUNT(*) AS games_release...	10 row(s) returned
3	17:24:23	SELECT Primary_Genre, COUNT(*) AS total_games FROM clean_games GROUP BY ...	20 row(s) returned
4	17:26:12	SELECT Title, Backlogs, Wishlist, ROUND(Backlogs / NULLIF(Wishlist, 0...	10 row(s) returned
5	17:27:24	SELECT YEAR(Release_Date) AS release_year, COUNT(*) AS total_releases F...	44 row(s) returned
6	17:28:30	SELECT FLOOR(Rating * 2) / 2 AS rating_bin, COUNT(*) AS game_count FRO...	9 row(s) returned
7	17:30:17	SELECT Title, Wishlist, Plays, Rating FROM clean_games ORDER BY Wishlist DESC ...	10 row(s) returned

```sql
-- 8. What's the average number of plays per genre?
    #Average number of plays per primary genre
SELECT
    Primary_Genre,
    ROUND(AVG(Plays), 0) AS avg_plays,
    COUNT(*) AS game_count
FROM clean_games
GROUP BY Primary_Genre
ORDER BY avg_plays DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Primary_Genre	avg_plays	game_count
MOBA	8750	2
Shooter	6398	58
Simulator	6110	12
Racing	5851	14
Platform	5439	34
Adventure	5229	713
Fighting	4908	37
Arcade	4349	43
Puzzle	4265	9

Result 9

Output

Action Output

#	Time	Action	Message
3	17:24:23	SELECT Primary_Genre, COUNT(*) AS total_games FROM clean_games GROUP BY ...	20 row(s) returned
4	17:26:12	SELECT Title, Backlogs, Wishlist, ROUND(Backlogs / NULLIF(Wishlist, 0...	10 row(s) returned
5	17:27:24	SELECT YEAR(Release_Date) AS release_year, COUNT(*) AS total_releases F...	44 row(s) returned
6	17:28:30	SELECT FLOOR(Rating * 2) / 2 AS rating_bin, COUNT(*) AS game_count FRO...	9 row(s) returned
7	17:30:17	SELECT Title, Wishlist, Plays, Rating FROM clean_games ORDER BY Wishlist DESC ...	10 row(s) returned
8	17:31:29	SELECT Primary_Genre, ROUND(AVG(Plays), 0) AS avg_plays, COUNT(*) A...	20 row(s) returned

```sql
-- 9. Which developer studios are the most productive and impactful?
    #Most productive and impactful developer studios
SELECT
    Team,
    COUNT(*) AS total_games,
    SUM(Plays) AS total_plays,
    ROUND(AVG(Rating), 2) AS avg_rating
FROM clean_games
WHERE Team <> 'Unknown'
GROUP BY Team
ORDER BY total_games DESC, total_plays DESC
LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Team	total_games	total_plays	avg_rating
Capcom	26	111177	3.67
Square Enix	21	59542	3.9
Nintendo, Game Freak	12	105100	3.78
Nintendo, Nintendo EAD	11	126600	3.73
Nintendo	11	96608	3.66
Intelligent Systems Co., Ltd., Nintendo	11	43068	3.6
Nintendo EAD, Nintendo	9	95600	3.83
Konami	9	16047	3.64
Ubisoft Montreal, Ubisoft Entertainment		67300	3.41

Konami

Result 10

Output

Action Output

#	Time	Action	Message
4	17:26:12	SELECT Title, Backlogs, Wishlist, ROUND(Backlogs / NULLIF(Wishlist, 0...	10 row(s) returned
5	17:27:24	SELECT YEAR(Release_Date) AS release_year, COUNT(*) AS total_releases F...	44 row(s) returned
6	17:28:30	SELECT FLOOR(Rating * 2) / 2 AS rating_bin, COUNT(*) AS game_count FRO...	9 row(s) returned
7	17:30:17	SELECT Title, Wishlist, Plays, Rating FROM clean_games ORDER BY Wishlist DESC ...	10 row(s) returned
8	17:31:29	SELECT Primary_Genre, ROUND(AVG(Plays), 0) AS avg_plays, COUNT(*) A...	20 row(s) returned
9	17:32:30	SELECT Team, COUNT(*) AS total_games, SUM(Plays) AS total_plays, R...	10 row(s) returned

```sql
-- ================================================================
--    vgsales.csv (Sales Data Only)
-- ================================================================
-- 10. Which region generates the most game sales?
   #Which region generates the most game sales?
SELECT
    ROUND(SUM(NA_Sales), 2) AS total_na_sales,
    ROUND(SUM(EU_Sales), 2) AS total_eu_sales,
    ROUND(SUM(JP_Sales), 2) AS total_jp_sales,
    ROUND(SUM(Other_Sales), 2) AS total_other_sales,
    ROUND(SUM(Global_Sales), 2) AS total_global_sales
FROM clean_vgsales;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_na_sales	total_eu_sales	total_jp_sales	total_other_sales	total_global_sales
4391.95	2433.41	1290.98	797.63	8918.56

Result 11

Output

Action Output

#	Time	Action	Message
5	17:27:24	SELECT YEAR(Release_Date) AS release_year, COUNT(*) AS total_releases F...	44 row(s) returned
6	17:28:30	SELECT FLOOR(Rating * 2) / 2 AS rating_bin, COUNT(*) AS game_count FRO...	9 row(s) returned
7	17:30:17	SELECT Title, Wishlist, Plays, Rating FROM clean_games ORDER BY Wishlist DESC ...	10 row(s) returned
8	17:31:29	SELECT Primary_Genre, ROUND(AVG(Plays), 0) AS avg_plays, COUNT(*) A...	20 row(s) returned
9	17:32:30	SELECT Team, COUNT(*) AS total_games, SUM(Plays) AS total_plays, R...	10 row(s) returned
10	17:34:14	SELECT ROUND(SUM(NA_Sales), 2) AS total_na_sales, ROUND(SUM(EU_Sal...	1 row(s) returned

```sql
-- 11. What are the best-selling platforms?
    #Best-selling platforms by global sales
SELECT
    Platform,
    COUNT(*) AS total_titles,
    ROUND(SUM(Global_Sales), 2) AS total_global_sales
FROM clean_vgsales
GROUP BY Platform
ORDER BY total_global_sales DESC
LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Platform	total_titles	total_global_sales
PS2	2161	1255.64
X360	1264	978.67
PS3	1327	957.35
WII	1324	926.69
DS	2163	822.49
PS	1196	730.66
GBA	822	318.5
PSP	1213	296.28
PS4	336	278.1

Result 12

Output

Action Output

#	Time	Action	Message
6	17:28:30	SELECT FLOOR(Rating * 2) / 2 AS rating_bin, COUNT(*) AS game_count FRO...	9 row(s) returned
7	17:30:17	SELECT Title, Wishlist, Plays, Rating FROM clean_games ORDER BY Wishlist DESC ...	10 row(s) returned
8	17:31:29	SELECT Primary_Genre, ROUND(AVG(Plays), 0) AS avg_plays, COUNT(*) A...	20 row(s) returned
9	17:32:30	SELECT Team, COUNT(*) AS total_games, SUM(Plays) AS total_plays, R...	10 row(s) returned
10	17:34:14	SELECT ROUND(SUM(NA_Sales), 2) AS total_na_sales, ROUND(SUM(EU_Sal...	1 row(s) returned
11	17:37:25	SELECT Platform, COUNT(*) AS total_titles, ROUND(SUM(Global_Sales), 2) ...	10 row(s) returned

```sql
-- 12. What's the trend of game releases and sales over years?
    #Trend of game releases and sales over years
SELECT
    Year,
    COUNT(*) AS titles_released,
    ROUND(SUM(Global_Sales), 2) AS global_sales_millions
FROM clean_vgsales
GROUP BY Year
ORDER BY Year ASC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Year	titles_released	global_sales_millions
1980	9	11.38
1981	46	35.77
1982	36	28.86
1983	17	16.79
1984	14	50.36
1985	14	53.94
1986	21	37.07
1987	16	21.74
1988	15	47.22

21.74

Result 13

Output

Action Output

#	Time	Action	Message
7	17:30:17	SELECT Title, Wishlist, Plays, Rating FROM clean_games ORDER BY Wishlist DESC ...	10 row(s) returned
8	17:31:29	SELECT Primary_Genre, ROUND(AVG(Plays), 0) AS avg_plays, COUNT(*) A...	20 row(s) returned
9	17:32:30	SELECT Team, COUNT(*) AS total_games, SUM(Plays) AS total_plays, R...	10 row(s) returned
10	17:34:14	SELECT ROUND(SUM(NA_Sales), 2) AS total_na_sales, ROUND(SUM(EU_Sal...	1 row(s) returned
11	17:37:25	SELECT Platform, COUNT(*) AS total_titles, ROUND(SUM(Global_Sales), 2) ...	10 row(s) returned
12	17:39:43	SELECT Year, COUNT(*) AS titles_released, ROUND(SUM(Global_Sales), 2)...	39 row(s) returned

```sql
-- 13. Who are the top publishers by sales?
    #Top publishers by global sales
SELECT
    Publisher,
    COUNT(*) AS titles,
    ROUND(SUM(Global_Sales), 2) AS total_global_sales
FROM clean_vgsales
WHERE Publisher <> 'Unknown'
GROUP BY Publisher
ORDER BY total_global_sales DESC
LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Publisher	titles	total_global_sales
Nintendo	702	1786.54
Electronic Arts	1348	1108.94
Activision	975	727.46
Sony Computer Entertainment	683	607.5
Ubisoft	921	474.72
Take-Two Interactive	413	399.54
THQ	715	340.77
Konami Digital Entertainment	832	283.64
Sega	639	272.99

Result 14

Output

Action Output

#	Time	Action	Message
8	17:31:29	SELECT Primary_Genre, ROUND(AVG(Plays), 0) AS avg_plays, COUNT(*) A...	20 row(s) returned
9	17:32:30	SELECT Team, COUNT(*) AS total_games, SUM(Plays) AS total_plays, R...	10 row(s) returned
10	17:34:14	SELECT ROUND(SUM(NA_Sales), 2) AS total_na_sales, ROUND(SUM(EU_Sal...	1 row(s) returned
11	17:37:25	SELECT Platform, COUNT(*) AS total_titles, ROUND(SUM(Global_Sales), 2) ...	10 row(s) returned
12	17:39:43	SELECT Year, COUNT(*) AS titles_released, ROUND(SUM(Global_Sales), 2)...	39 row(s) returned
13	17:41:32	SELECT Publisher, COUNT(*) AS titles, ROUND(SUM(Global_Sales), 2) AS t...	10 row(s) returned

```sql
-- 14. Which games are the top 10 best-sellers globally?
    #Top 10 best-selling games globally
SELECT
    'Rank', 'Name', Platform,
    'Year',
    Genre,
    Publisher,
    Global_Sales
FROM clean_vgsales
ORDER BY Global_Sales DESC
LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Rank	Name	Platform	Year	Genre	Publisher	Global_Sales
Rank	Name	WII	Year	Sports	Nintendo	82.74
Rank	Name	NES	Year	Platform	Nintendo	40.24
Rank	Name	WII	Year	Racing	Nintendo	35.82
Rank	Name	WII	Year	Sports	Nintendo	33
Rank	Name	GB	Year	Role-Playing	Nintendo	31.37
Rank	Name	GB	Year	Puzzle	Nintendo	30.26
Rank	Name	DS	Year	Platform	Nintendo	30.01
Rank	Name	WII	Year	Misc	Nintendo	29.02
Rank	Name	WII	Year	Platform	Nintendo	28.62

Result 16

Output

Action Output

#	Time	Action	Message
11	17:37:25	SELECT Platform, COUNT(*) AS total_titles, ROUND(SUM(Global_Sales), 2) ...	10 row(s) returned
12	17:39:43	SELECT Year, COUNT(*) AS titles_released, ROUND(SUM(Global_Sales), 2)...	39 row(s) returned
13	17:41:32	SELECT Publisher, COUNT(*) AS titles, ROUND(SUM(Global_Sales), 2) AS t...	10 row(s) returned
14	17:42:41	SELECT Rank, Name, Platform, Year, Genre, Publisher, Global_...	Error Code: 1064. You
15	17:44:59	SELECT 'Rank', 'Name', Platform, 'Year', Genre, Publisher, Global_Sale...	10 row(s) returned
16	17:45:38	SELECT 'Rank', 'Name', Platform, 'Year', Genre, Publisher, Global_Sale...	10 row(s) returned

```sql
-- 15. How do regional sales compare for specific platforms?
    #Regional sales comparison for specific top platforms (e.g., PS2, X360, PS3, Wii)
SELECT
    Platform,
    ROUND(SUM(NA_Sales), 2) AS na_sales,
    ROUND(SUM(EU_Sales), 2) AS eu_sales,
    ROUND(SUM(JP_Sales), 2) AS jp_sales,
    ROUND(SUM(Other_Sales), 2) AS other_sales,
    ROUND(SUM(Global_Sales), 2) AS total_sales
FROM clean_vgsales
GROUP BY Platform
ORDER BY total_sales DESC
LIMIT 8;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Platform	na_sales	eu_sales	jp_sales	other_sales	total_sales
PS2	583.84	339.29	139.2	193.44	1255.64
X360	600.05	280.41	12.41	85.44	978.67
PS3	392.26	343.22	79.99	141.93	957.35
WII	507.71	268.38	69.33	80.61	926.69
DS	390.71	194.65	175.57	60.53	822.49
PS	336.51	213.6	139.82	40.91	730.66
GBA	187.54	75.25	47.33	7.73	318.5
PSP	108.99	68.25	76.79	42.19	296.28

Result 18

Output

Action Output

#	Time	Action	Message
13	17:41:32	SELECT Publisher, COUNT(*) AS titles, ROUND(SUM(Global_Sales), 2) AS t...	10 row(s) returned
14	17:42:41	SELECT Rank, Name, Platform, Year, Genre, Publisher, Global_...	Error Code: 1064. You
15	17:44:59	SELECT 'Rank', 'Name', Platform, 'Year', Genre, Publisher, Global_Sale...	10 row(s) returned
16	17:45:38	SELECT 'Rank', 'Name', Platform, 'Year', Genre, Publisher, Global_Sale...	10 row(s) returned
17	17:47:12	SELECT Platform, ROUND(SUM(NA_Sales), 2) AS na_sales, ROUND(SUM(E...	31 row(s) returned
18	17:49:14	SELECT Platform, ROUND(SUM(NA_Sales), 2) AS na_sales, ROUND(SUM(E...	8 row(s) returned

```sql
-- 16. How has the market evolved by platform over time?
    #How the market evolved by platform over time (Decade / Multi-year view)
SELECT
    Year,
    Platform,
    ROUND(SUM(Global_Sales), 2) AS total_platform_sales
FROM clean_vgsales
WHERE Year >= 2000
GROUP BY Year, Platform
ORDER BY Year DESC, total_platform_sales DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Year	Platform	total_platform_sales
2020	DS	0.29
2017	PS4	0.03
2017	PSV	0.02
2016	PS4	39.25
2016	XONE	12.37
2016	3DS	6.6
2016	PSV	3.4
2016	WIIU	3.29
2016	PC	2.6

Result 19

Output

Action Output

#	Time	Action	Message
14	17:42:41	SELECT Rank, Name, Platform, Year, Genre, Publisher, Global_...	Error Code: 1064. You
15	17:44:59	SELECT 'Rank', 'Name', Platform, 'Year', Genre, Publisher, Global_Sale...	10 row(s) returned
16	17:45:38	SELECT 'Rank', 'Name', Platform, 'Year', Genre, Publisher, Global_Sale...	10 row(s) returned
17	17:47:12	SELECT Platform, ROUND(SUM(NA_Sales), 2) AS na_sales, ROUND(SUM(E...	31 row(s) returned
18	17:49:14	SELECT Platform, ROUND(SUM(NA_Sales), 2) AS na_sales, ROUND(SUM(E...	8 row(s) returned
19	17:51:08	SELECT Year, Platform, ROUND(SUM(Global_Sales), 2) AS total_platform_s...	159 row(s) returned

```sql
-- 17. What are the regional genre preferences?
    #Regional genre preferences
SELECT
    Genre,
    ROUND(SUM(NA_Sales), 2) AS na_sales,
    ROUND(SUM(EU_Sales), 2) AS eu_sales,
    ROUND(SUM(JP_Sales), 2) AS jp_sales,
    ROUND(SUM(Global_Sales), 2) AS global_sales
FROM clean_vgsales
GROUP BY Genre
ORDER BY global_sales DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Genre	na_sales	eu_sales	jp_sales	global_sales
Action	877.83	525	159.95	1751.18
Sports	683.35	376.84	135.37	1330.92
Shooter	582.6	313.27	38.26	1037.35
Role-Playing	327.28	188.06	352.31	927.37
Platform	447.05	201.15	130.77	830.89
Misc	410.24	215.98	107.76	809.96
Racing	358.42	238.16	56.67	730.67
Fighting	223.59	101.32	87.35	448.91
Simulation	183.31	113.38	63.7	392.2

Result 21

Output

Action Output

#	Time	Action	Message
16	17:45:38	SELECT 'Rank', 'Name', Platform, 'Year', Genre, Publisher, Global_Sale...	10 row(s) returned
17	17:47:12	SELECT Platform, ROUND(SUM(NA_Sales), 2) AS na_sales, ROUND(SUM(E...	31 row(s) returned
18	17:49:14	SELECT Platform, ROUND(SUM(NA_Sales), 2) AS na_sales, ROUND(SUM(E...	8 row(s) returned
19	17:51:08	SELECT Year, Platform, ROUND(SUM(Global_Sales), 2) AS total_platform_s...	159 row(s) returned
20	17:52:28	SELECT Genre, ROUND(SUM(NA_Sales), 2) AS na_sales, ROUND(SUM(EU...	12 row(s) returned
21	17:52:54	SELECT Genre, ROUND(SUM(NA_Sales), 2) AS na_sales, ROUND(SUM(EU...	12 row(s) returned

```sql
-- 18. What's the yearly sales change per region?
    #Yearly sales change per region
SELECT
    Year,
    ROUND(SUM(NA_Sales), 2) AS na_yearly_sales,
    ROUND(SUM(EU_Sales), 2) AS eu_yearly_sales,
    ROUND(SUM(JP_Sales), 2) AS jp_yearly_sales
FROM clean_vgsales
GROUP BY Year
ORDER BY Year ASC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Year	na_yearly_sales	eu_yearly_sales	jp_yearly_sales
1980	10.59	0.67	0
1981	33.4	1.96	0
1982	26.92	1.65	0
1983	7.76	0.8	8.1
1984	33.28	2.1	14.27
1985	33.73	4.74	19.43
1986	12.5	2.84	19.81
1987	8.46	1.41	11.63
1988	23.87	6.59	15.76

Result 22

Output

Action Output

#	Time	Action	Message
17	17:47:12	SELECT Platform, ROUND(SUM(NA_Sales), 2) AS na_sales, ROUND(SUM(E...	31 row(s) returned
18	17:49:14	SELECT Platform, ROUND(SUM(NA_Sales), 2) AS na_sales, ROUND(SUM(E...	8 row(s) returned
19	17:51:08	SELECT Year, Platform, ROUND(SUM(Global_Sales), 2) AS total_platform_s...	159 row(s) returned
20	17:52:28	SELECT Genre, ROUND(SUM(NA_Sales), 2) AS na_sales, ROUND(SUM(EU...	12 row(s) returned
21	17:52:54	SELECT Genre, ROUND(SUM(NA_Sales), 2) AS na_sales, ROUND(SUM(EU...	12 row(s) returned
22	17:54:05	SELECT Year, ROUND(SUM(NA_Sales), 2) AS na_yearly_sales, ROUND(SU...	39 row(s) returned

```sql
-- 19. What is the average sales per publisher?
    #Average sales per publisher (minimum 10 releases)
SELECT
    Publisher,
    COUNT(*) AS total_games,
    ROUND(AVG(Global_Sales), 3) AS avg_sales_per_title,
    ROUND(SUM(Global_Sales), 2) AS total_sales
FROM clean_vgsales
WHERE Publisher <> 'Unknown'
GROUP BY Publisher
HAVING COUNT(*) >= 10
ORDER BY avg_sales_per_title DESC
LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Publisher	total_games	avg_sales_per_title	total_sales
Nintendo	702	2.545	1786.54
Sony Computer Entertainment Europe	15	1.593	23.89
Microsoft Game Studios	189	1.3	245.79
Bethesda Softworks	71	1.157	82.14
Enix Corporation	30	1.125	33.74
SquareSoft	52	1.109	57.65
LucasArts	90	0.97	87.65
Take-Two Interactive	413	0.967	399.54
Hasbro Interactive	16	0.951	15.22

Result 23

Output

Action Output

#	Time	Action	Message
18	17:49:14	SELECT Platform, ROUND(SUM(NA_Sales), 2) AS na_sales, ROUND(SUM(E...	8 row(s) returned
19	17:51:08	SELECT Year, Platform, ROUND(SUM(Global_Sales), 2) AS total_platform_s...	159 row(s) returned
20	17:52:28	SELECT Genre, ROUND(SUM(NA_Sales), 2) AS na_sales, ROUND(SUM(EU...	12 row(s) returned
21	17:52:54	SELECT Genre, ROUND(SUM(NA_Sales), 2) AS na_sales, ROUND(SUM(EU...	12 row(s) returned
22	17:54:05	SELECT Year, ROUND(SUM(NA_Sales), 2) AS na_yearly_sales, ROUND(SU...	39 row(s) returned
23	17:55:37	SELECT Publisher, COUNT(*) AS total_games, ROUND(AVG(Global_Sales), 3...	10 row(s) returned

```sql
-- 20. What are the top 5 best-selling games per platform?
    #Top 5 best-selling games per top platform (Using Window Function)
WITH RankedPlatformGames AS (
    SELECT
        Name,
        Platform,
        Global_Sales,
        ROW_NUMBER() OVER(PARTITION BY Platform ORDER BY Global_Sales DESC) AS platform_rank
    FROM clean_vgsales
)
SELECT Platform, platform_rank, Name, Global_Sales
FROM RankedPlatformGames
WHERE platform_rank <= 5 AND Platform IN ('PS2', 'X360', 'PS3', 'WII', 'DS', 'PC')
ORDER BY Platform, platform_rank;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Platform	platform_rank	Name	Global_Sales
DS	1	New Super Mario Bros.	30.01
DS	2	Nintendogs	24.76
DS	3	Mario Kart DS	23.42
DS	4	Brain Age: Train Your Brain in Minutes a Day	20.22
DS	5	Pokemon Diamond/Pokemon Pearl	18.36
PC	1	The Sims 3	8.11
PC	2	World of Warcraft	6.28
PC	3	Diablo III	5.2
PC	4	Microsoft Flight Simulator	

Microsoft Flight Simulator

Result 24

Output

Action Output

#	Time	Action	Message
19	17:51:08	SELECT Year, Platform, ROUND(SUM(Global_Sales), 2) AS total_platform_s...	159 row(s) returned
20	17:52:28	SELECT Genre, ROUND(SUM(NA_Sales), 2) AS na_sales, ROUND(SUM(EU...	12 row(s) returned
21	17:52:54	SELECT Genre, ROUND(SUM(NA_Sales), 2) AS na_sales, ROUND(SUM(EU...	12 row(s) returned
22	17:54:05	SELECT Year, ROUND(SUM(NA_Sales), 2) AS na_yearly_sales, ROUND(SU...	39 row(s) returned
23	17:55:37	SELECT Publisher, COUNT(*) AS total_games, ROUND(AVG(Global_Sales), 3...	10 row(s) returned
24	17:57:17	WITH RankedPlatformGames AS (SELECT Name, Platform, Globa...	30 row(s) returned

```sql
-- ============================================================
--       Merged Dataset (Sales + Engagement + Ratings)
-- ============================================================
-- 21. Which game genres generate the most global sales?
    #Game genres generating the most global sales in merged data
SELECT
    Genre,
    COUNT(*) AS title_count,
    ROUND(SUM(Global_Sales), 2) AS total_global_sales,
    ROUND(AVG(Rating), 2) AS avg_user_rating
FROM clean_merged_vgsales
GROUP BY Genre
ORDER BY total_global_sales DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Genre	title_count	total_global_sales	avg_user_rating
Action	309	554.09	3.55
Shooter	157	398.78	3.47
Platform	116	376.96	3.59
Role-Playing	141	223.46	3.81
Racing	24	126.37	3.38
Sports	24	113.38	3.46
Fighting	40	68.47	3.35
Puzzle	16	62.92	3.79
Misc	30	60.18	3.69

3.79

Result 25

Output

Action Output

#	Time	Action	Message
20	17:52:28	SELECT Genre, ROUND(SUM(NA_Sales), 2) AS na_sales, ROUND(SUM(EU...	12 row(s) returned
21	17:52:54	SELECT Genre, ROUND(SUM(NA_Sales), 2) AS na_sales, ROUND(SUM(EU...	12 row(s) returned
22	17:54:05	SELECT Year, ROUND(SUM(NA_Sales), 2) AS na_yearly_sales, ROUND(SU...	39 row(s) returned
23	17:55:37	SELECT Publisher, COUNT(*) AS total_games, ROUND(AVG(Global_Sales), 3...	10 row(s) returned
24	17:57:17	WITH RankedPlatformGames AS (SELECT Name, Platform, Globa...	30 row(s) returned
25	18:03:30	SELECT Genre, COUNT(*) AS title_count, ROUND(SUM(Global_Sales), 2) A...	12 row(s) returned

```sql
-- 22. How does user rating affect global sales?
    #How user rating affects global sales (Rating Tier breakdown)
SELECT
    CASE
        WHEN Rating >= 4.5 THEN '4.5 - 5.0 (Masterpiece)'
        WHEN Rating >= 4.0 THEN '4.0 - 4.4 (Great)'
        WHEN Rating >= 3.5 THEN '3.5 - 3.9 (Good)'
        WHEN Rating >= 3.0 THEN '3.0 - 3.4 (Average)'
        ELSE 'Below 3.0 (Poor)'
    END AS rating_tier,
    COUNT(*) AS number_of_titles,
    ROUND(AVG(Global_Sales), 2) AS avg_sales,
    ROUND(SUM(Global_Sales), 2) AS total_sales
FROM clean_merged_vgsales
ORDER BY avg_sales DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

rating_tier	number_of_titles	avg_sales	total_sales
3.5 - 3.9 (Good)	361	2.72	980.95
4.0 - 4.4 (Great)	237	2.26	535.85
Below 3.0 (Poor)	90	1.66	149.57
4.5 - 5.0 (Masterpiece)	8	1.61	12.9
3.0 - 3.4 (Average)	249	1.54	383.24

Result 26

Output

Action Output

#	Time	Action	Message
21	17:52:54	SELECT Genre, ROUND(SUM(NA_Sales), 2) AS na_sales, ROUND(SUM(EU...	12 row(s) returned
22	17:54:05	SELECT Year, ROUND(SUM(NA_Sales), 2) AS na_yearly_sales, ROUND(SU...	39 row(s) returned
23	17:55:37	SELECT Publisher, COUNT(*) AS total_games, ROUND(AVG(Global_Sales), 3...	10 row(s) returned
24	17:57:17	WITH RankedPlatformGames AS (SELECT Name, Platform, Globa...	30 row(s) returned
25	18:03:30	SELECT Genre, COUNT(*) AS title_count, ROUND(SUM(Global_Sales), 2) A...	12 row(s) returned
26	18:05:15	SELECT CASE WHEN Rating >= 4.5 THEN '4.5 - 5.0 (Masterpiece)' WH...	5 row(s) returned

```sql
-- 23. Which platforms have the most games with high ratings (e.g., above 4)?
    #Platforms with the most games rated above 4.0
SELECT
    Platform,
    COUNT(*) AS high_rated_games,
    ROUND(AVG(Rating), 2) AS avg_platform_rating
FROM clean_merged_vgsales
WHERE Rating >= 4.0
GROUP BY Platform
ORDER BY high_rated_games DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Platform	high_rated_games	avg_platform_rating
PS2	29	4.21
PC	26	4.16
X360	25	4.11
PS3	24	4.16
PS	16	4.19
PS4	14	4.14
DS	12	4.23
3DS	12	4.13
SNES	11	4.17

Result 27

Output

Action Output

#	Time	Action	Message
22	17:54:05	SELECT Year, ROUND(SUM(NA_Sales), 2) AS na_yearly_sales, ROUND(SU...	39 row(s) returned
23	17:55:37	SELECT Publisher, COUNT(*) AS total_games, ROUND(AVG(Global_Sales), 3...	10 row(s) returned
24	17:57:17	WITH RankedPlatformGames AS (SELECT Name, Platform, Globa...	30 row(s) returned
25	18:03:30	SELECT Genre, COUNT(*) AS title_count, ROUND(SUM(Global_Sales), 2) A...	12 row(s) returned
26	18:05:15	SELECT CASE WHEN Rating >= 4.5 THEN '4.5 - 5.0 (Masterpiece)' WH...	5 row(s) returned
27	18:06:54	SELECT Platform, COUNT(*) AS high_rated_games, ROUND(AVG(Rating), 2)...	21 row(s) returned

```sql
-- 24. What's the trend of releases and sales over time?
    #Trend of releases, ratings, and sales over time
SELECT
    Year,
    COUNT(*) AS titles_released,
    ROUND(AVG(Rating), 2) AS avg_rating,
    ROUND(SUM(Global_Sales), 2) AS total_sales
FROM clean_merged_vgsales
GROUP BY Year
ORDER BY Year ASC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Year	titles_released	avg_rating	total_sales
1981	1	3.6	1.65
1982	1	3.4	7.81
1984	1	3.4	1.22
1985	1	3.5	40.24
1986	2	3.3	7.74
1987	2	2.7	5.19
1988	4	3.7	31.83
1989	2	3.85	31.11
1990	3	3.57	23.18

Result 28

Output

Action Output

#	Time	Action	Message
23	17:55:37	SELECT Publisher, COUNT(*) AS total_games, ROUND(AVG(Global_Sales), 3...	10 row(s) returned
24	17:57:17	WITH RankedPlatformGames AS (SELECT Name, Platform, Globa...	30 row(s) returned
25	18:03:30	SELECT Genre, COUNT(*) AS title_count, ROUND(SUM(Global_Sales), 2) A...	12 row(s) returned
26	18:05:15	SELECT CASE WHEN Rating >= 4.5 THEN '4.5 - 5.0 (Masterpiece)' WH...	5 row(s) returned
27	18:06:54	SELECT Platform, COUNT(*) AS high_rated_games, ROUND(AVG(Rating), 2)...	21 row(s) returned
28	18:08:23	SELECT Year, COUNT(*) AS titles_released, ROUND(AVG(Rating), 2) AS av...	35 row(s) returned

```sql
-- 25. Do highly wishlisted games lead to more sales?
    #Do highly wishlisted games lead to more sales?
SELECT
    CASE
        WHEN Wishlist >= 3000 THEN '3k+ Wishlists'
        WHEN Wishlist >= 1000 THEN '1k - 3k Wishlists'
        WHEN Wishlist >= 500  THEN '500 - 1k Wishlists'
        ELSE '< 500 Wishlists'
    END AS wishlist_bracket,
    COUNT(*) AS total_games,
    ROUND(AVG(Global_Sales), 2) AS avg_global_sales,
    ROUND(SUM(Global_Sales), 2) AS total_sales
FROM clean_merged_vgsales
GROUP BY wishlist_bracket
ORDER BY avg_global_sales DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

wishlist_bracket	total_games	avg_global_sales	total_sales
3k+ Wishlists	1	2.38	2.38
< 500 Wishlists	607	2.28	1383.24
1k - 3k Wishlists	103	2.25	231.32
500 - 1k Wishlists	234	1.9	445.57

Result 29

Output

Action Output

#	Time	Action	Message
24	17:57:17	WITH RankedPlatformGames AS (SELECT Name, Platform, Globa...	30 row(s) returned
25	18:03:30	SELECT Genre, COUNT(*) AS title_count, ROUND(SUM(Global_Sales), 2) A...	12 row(s) returned
26	18:05:15	SELECT CASE WHEN Rating >= 4.5 THEN '4.5 - 5.0 (Masterpiece)' WH...	5 row(s) returned
27	18:06:54	SELECT Platform, COUNT(*) AS high_rated_games, ROUND(AVG(Rating), 2)...	21 row(s) returned
28	18:08:23	SELECT Year, COUNT(*) AS titles_released, ROUND(AVG(Rating), 2) AS av...	35 row(s) returned
29	18:10:01	SELECT CASE WHEN Wishlist >= 3000 THEN '3k+ Wishlists' WHEN W...	4 row(s) returned

```sql
-- 26. Which genres have the highest engagement but lowest sales?
    #Genres with highest engagement (Plays) but lowest sales
SELECT
    Genre,
    ROUND(AVG(Plays), 0) AS avg_plays,
    ROUND(AVG(Global_Sales), 2) AS avg_sales,
    ROUND(AVG(Plays) / NULLIF(AVG(Global_Sales), 0), 0) AS engagement_to_sales_ratio
FROM clean_merged_vgsales
GROUP BY Genre
ORDER BY engagement_to_sales_ratio DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Genre	avg_plays	avg_sales	engagement_to_sales_ratio
Adventure	6047	0.62	9778
Strategy	5855	0.79	7411
Misc	10307	2.01	5138
Action	7298	1.79	4070
Role-Playing	6045	1.58	3814
Shooter	8483	2.54	3340
Fighting	4710	1.71	2752
Platform	6961	3.25	2142
Simulation	3385	2.23	1520

Result 30

Output

Action Output

#	Time	Action	Message
25	18:03:30	SELECT Genre, COUNT(*) AS title_count, ROUND(SUM(Global_Sales), 2) A...	12 row(s) returned
26	18:05:15	SELECT CASE WHEN Rating >= 4.5 THEN '4.5 - 5.0 (Masterpiece)' WH...	5 row(s) returned
27	18:06:54	SELECT Platform, COUNT(*) AS high_rated_games, ROUND(AVG(Rating), 2)...	21 row(s) returned
28	18:08:23	SELECT Year, COUNT(*) AS titles_released, ROUND(AVG(Rating), 2) AS av...	35 row(s) returned
29	18:10:01	SELECT CASE WHEN Wishlist >= 3000 THEN '3k+ Wishlists' WHEN W...	4 row(s) returned
30	18:12:28	SELECT Genre, ROUND(AVG(Plays), 0) AS avg_plays, ROUND(AVG(Global_...	12 row(s) returned

```sql
-- 27. Do highly listed games (wishlist/backlogs) correlate with better ratings?
    #Do highly listed games (Wishlist + Backlogs) correlate with better ratings?
SELECT
    CASE
        WHEN (Wishlist + Backlogs) >= 5000 THEN 'Very High (5000+)'
        WHEN (Wishlist + Backlogs) >= 2000 THEN 'High (2000-4999)'
        WHEN (Wishlist + Backlogs) >= 500  THEN 'Moderate (500-1999)'
        ELSE 'Low (<500)'
    END AS listed_status,
    COUNT(*) AS game_count,
    ROUND(AVG(Rating), 2) AS avg_rating
FROM clean_merged_vgsales
GROUP BY listed_status
ORDER BY avg_rating DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

listed_status	game_count	avg_rating
Very High (5000+)	46	4.13
High (2000-4999)	276	3.83
Moderate (500-1999)	434	3.5
Low (<500)	189	3.26

Result 31

Output

Action Output

#	Time	Action	Message
26	18:05:15	SELECT CASE WHEN Rating >= 4.5 THEN '4.5 - 5.0 (Masterpiece)' WH...	5 row(s) returned
27	18:06:54	SELECT Platform, COUNT(*) AS high_rated_games, ROUND(AVG(Rating), 2)...	21 row(s) returned
28	18:08:23	SELECT Year, COUNT(*) AS titles_released, ROUND(AVG(Rating), 2) AS av...	35 row(s) returned
29	18:10:01	SELECT CASE WHEN Wishlist >= 3000 THEN '3k+ Wishlists' WHEN W...	4 row(s) returned
30	18:12:28	SELECT Genre, ROUND(AVG(Plays), 0) AS avg_plays, ROUND(AVG(Global_...	12 row(s) returned
31	18:13:43	SELECT CASE WHEN (Wishlist + Backlogs) >= 5000 THEN 'Very High (5000...	4 row(s) returned

```sql
-- 28. How does user engagement differ across genres?
    #How user engagement differs across genres
SELECT
    Genre,
    ROUND(AVG(Plays), 0) AS avg_plays,
    ROUND(AVG(Backlogs), 0) AS avg_backlogs,
    ROUND(AVG(Wishlist), 0) AS avg_wishlist
FROM clean_merged_vgsales
GROUP BY Genre
ORDER BY avg_plays DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Genre	avg_plays	avg_backlogs	avg_wishlist
Misc	10307	555	175
Shooter	8483	1303	430
Action	7298	1429	573
Platform	6961	1179	434
Adventure	6047	1208	524
Role-Playing	6045	1887	751
Strategy	5855	583	213
Racing	4900	283	118
Fighting	4710	287	115

118

Result 32

Output

Action Output

#	Time	Action	Message
27	18:06:54	SELECT Platform, COUNT(*) AS high_rated_games, ROUND(AVG(Rating), 2)...	21 row(s) returned
28	18:08:23	SELECT Year, COUNT(*) AS titles_released, ROUND(AVG(Rating), 2) AS av...	35 row(s) returned
29	18:10:01	SELECT CASE WHEN Wishlist >= 3000 THEN '3k+ Wishlists' WHEN W...	4 row(s) returned
30	18:12:28	SELECT Genre, ROUND(AVG(Plays), 0) AS avg_plays, ROUND(AVG(Global_...	12 row(s) returned
31	18:13:43	SELECT CASE WHEN (Wishlist + Backlogs) >= 5000 THEN 'Very High (5000...	4 row(s) returned
32	18:15:57	SELECT Genre, ROUND(AVG(Plays), 0) AS avg_plays, ROUND(AVG(Backlog...	12 row(s) returned

```sql
-- 29. What are the top-performing combinations of Genre + Platform?
    #Top-performing combinations of Genre + Platform
SELECT
    Genre,
    Platform,
    COUNT(*) AS titles,
    ROUND(SUM(Global_Sales), 2) AS total_sales,
    ROUND(AVG(Rating), 2) AS avg_rating
FROM clean_merged_vgsales
GROUP BY Genre, Platform
HAVING COUNT(*) >= 3
ORDER BY total_sales DESC
LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Genre	Platform	titles	total_sales	avg_rating
Shooter	X360	34	162.56	3.49
Action	PS3	52	140.25	3.48
Shooter	PS3	27	106.19	3.4
Action	PS2	32	100.86	3.72
Action	X360	49	98.71	3.47
Sports	WII	4	85.53	3.55
Platform	NES	8	70.79	3.42
Platform	WII	9	65.47	3.58
Platform	DS	9	50.14	74

50.14

Result 33

Output

Action Output

#	Time	Action	Message
28	18:08:23	SELECT Year, COUNT(*) AS titles_released, ROUND(AVG(Rating), 2) AS av...	35 row(s) returned
29	18:10:01	SELECT CASE WHEN Wishlist >= 3000 THEN '3k+ Wishlists' WHEN W...	4 row(s) returned
30	18:12:28	SELECT Genre, ROUND(AVG(Plays), 0) AS avg_plays, ROUND(AVG(Global_...	12 row(s) returned
31	18:13:43	SELECT CASE WHEN (Wishlist + Backlogs) >= 5000 THEN 'Very High (5000...	4 row(s) returned
32	18:15:57	SELECT Genre, ROUND(AVG(Plays), 0) AS avg_plays, ROUND(AVG(Backlog...	12 row(s) returned
33	18:17:10	SELECT Genre, Platform, COUNT(*) AS titles, ROUND(SUM(Global_Sales...	10 row(s) returned

```sql
-- 30. What does a regional sales heatmap by genre reveal?
    #Regional sales breakdown by genre (Heatmap matrix query)
SELECT
    Genre,
    ROUND(SUM(NA_Sales), 2) AS na_sales,
    ROUND(SUM(EU_Sales), 2) AS eu_sales,
    ROUND(SUM(JP_Sales), 2) AS jp_sales,
    ROUND(SUM(Other_Sales), 2) AS other_sales,
    ROUND(SUM(Global_Sales), 2) AS global_sales
FROM clean_merged_vgsales
GROUP BY Genre
ORDER BY global_sales DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Genre	na_sales	eu_sales	jp_sales	other_sales	global_sales
Action	263.95	177.4	38.38	74.13	554.09
Shooter	215.81	130.69	9.22	43.04	398.78
Platform	199.74	90.66	63.11	23.41	376.96
Role-Playing	92.2	60.93	52.81	17.5	223.46
Racing	59.17	41.12	16.91	9.22	126.37
Sports	52.84	43.33	4.72	12.51	113.38
Fighting	35.89	15.48	11.08	6.06	68.47
Puzzle	41.12	9.52	10.08	2.22	62.92
Misc	33.45	14.58	4.8	7.4	60.18

Result 34

Output

Action Output

#	Time	Action	Message
29	18:10:01	SELECT CASE WHEN Wishlist >= 3000 THEN '3k+ Wishlists' WHEN W...	4 row(s) returned
30	18:12:28	SELECT Genre, ROUND(AVG(Plays), 0) AS avg_plays, ROUND(AVG(Global_...	12 row(s) returned
31	18:13:43	SELECT CASE WHEN (Wishlist + Backlogs) >= 5000 THEN 'Very High (5000...	4 row(s) returned
32	18:15:57	SELECT Genre, ROUND(AVG(Plays), 0) AS avg_plays, ROUND(AVG(Backlog...	12 row(s) returned
33	18:17:10	SELECT Genre, Platform, COUNT(*) AS titles, ROUND(SUM(Global_Sales...	10 row(s) returned
34	18:18:42	SELECT Genre, ROUND(SUM(NA_Sales), 2) AS na_sales, ROUND(SUM(EU...	12 row(s) returned

Video Game Sales And Engagement Analytics

Commercial sales Performance

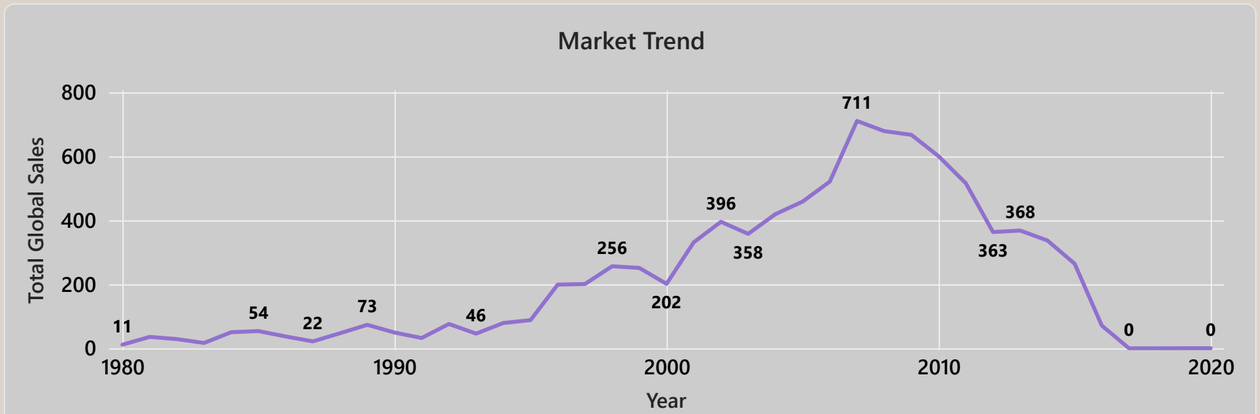
Year: All | Platform: All | Genre: All

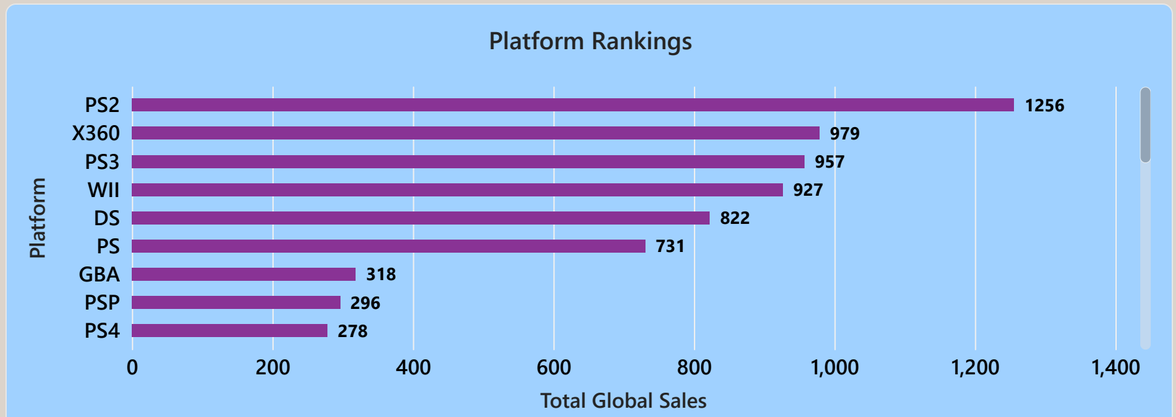
Total Global Sales
8.92K

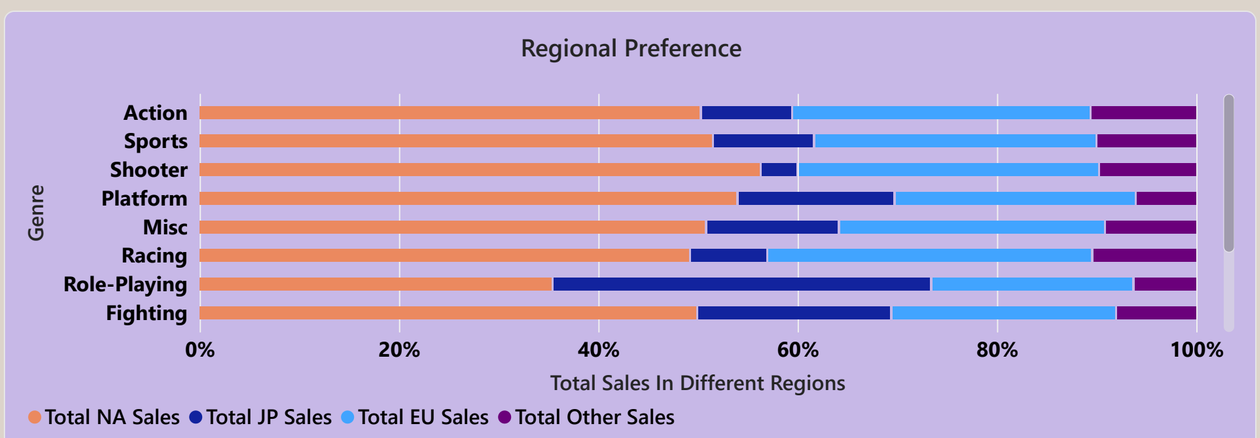
Total EU Sales
2.43K

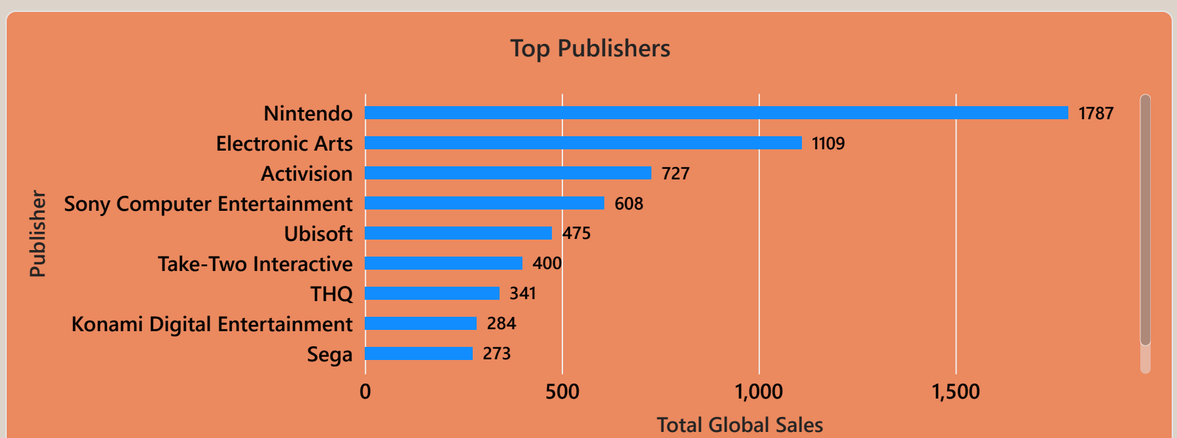
Total JP Sales
1.29K

Total NA Sales
4.39K









Gamer Engagement And Metadata

Year: All | Platform: All | Genre: All

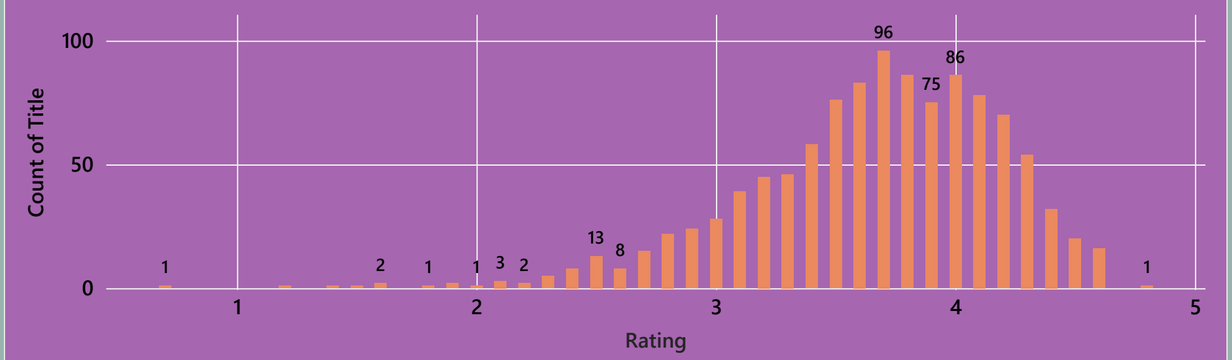
Avg User Rating
3.66

Total Plays
5M

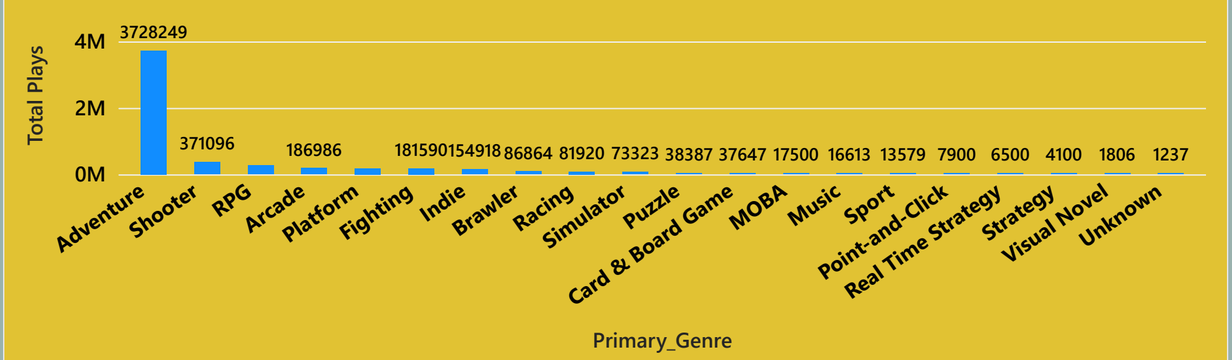
Total Wishlist
671K

Backlog to Wishlist Ratio
1.91

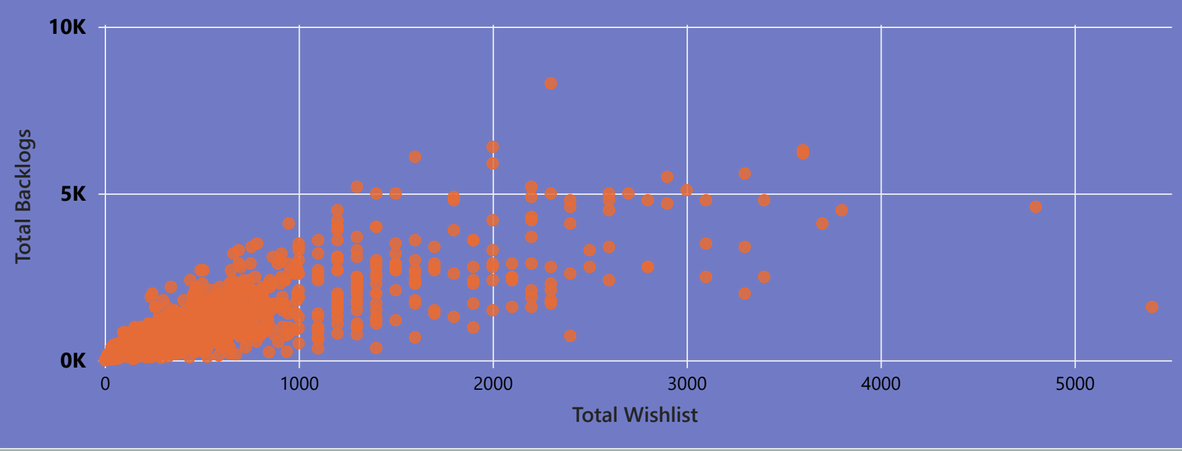
Rating Spread



Genre Activity



Backlog vs Wishlist



Top Studio Quality

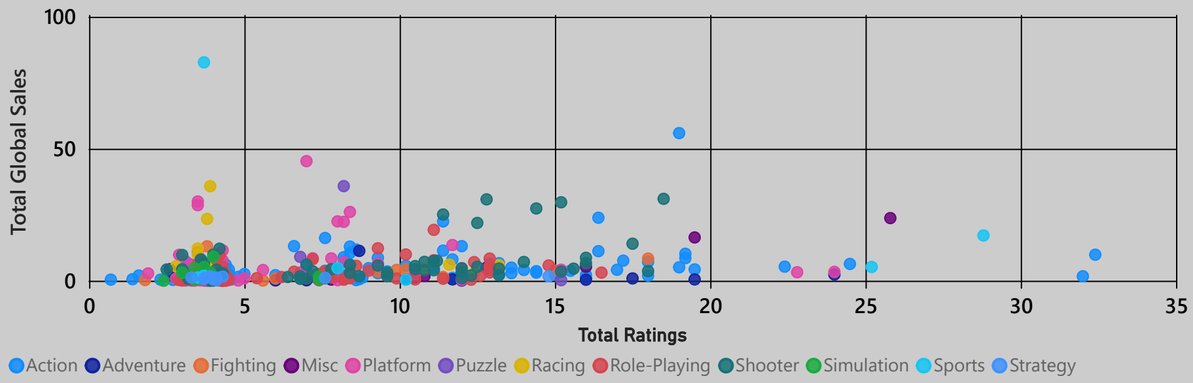
Team	Avg User Rating	First Title
Orange_Juice, Fruitbat Factory	3.40	100% Orange Juice
Vanillaware, Atlus USA	4.40	13 Sentinels: Aegis Rim
Chunsoft, Sega	4.20	428: Shibuya Scramble
Humble Bundle, Gears for Breakfast	4.00	A Hat in Time
Focus Entertainment, Asobo Studio	3.70	A Plague Tale: Innocence
adamgryu, Whippoorwill	4.10	A Short Hike
Electronic Arts, Hazelight Studios	3.60	A Way Out
505 Games, Giant Squid	3.40	Abzu
Total	3.66	100% Orange Juice

Merged Correlation And Strategic Insights

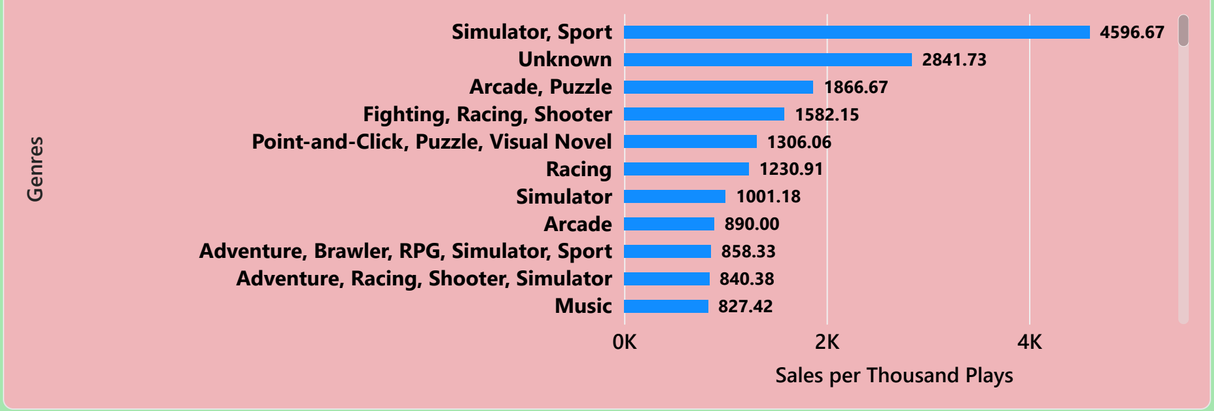
Year: All | Platform: All | Genre: All

Merged Global Sales	Merged Avg Rating	High Rated Games Count	Sales per Thousand Plays
2.06K	3.58	143	316.01

Rating vs Sales



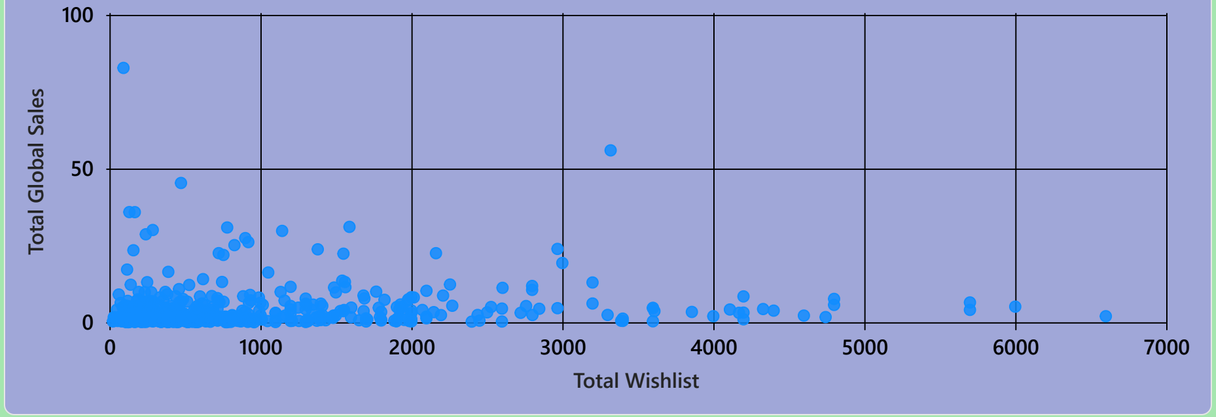
Conversion Outliers



Genre Heatmap

Platform	2600		3DS	
Genres	Merged Global Sales	Count of Title	Merged Global Sales	Coun
Simulator			9.09	
Simulator, Sport				
Simulator, Strategy, Tactical, Turn Based Strategy				
Simulator, Strategy, Turn Based Strategy				
Sport			0.25	
Unknown				
Total	9.46	2	78.47	

Wishlist vs Sales



Advance Wars

2600

Total Rating

3.38K

Total Global_Sales

2.06K

Total Plays

7M

Total Wishlist

456K

Executive Summary

This project analysed historical video game commercial performance and player engagement by combining retail sales records (vgsales.csv) with modern community engagement metrics (games.csv). The synthesis reveals key dynamics governing commercial success, consumer intent, regional divergence, and the relationship between critical reception and revenue.

Key Data Insights

1. Commercial Sales Dynamics (vgsales)

- **Regional Dominance:** North America accounts for **49.2%** of all historical global revenue ($4,392M), followed by Europe at **27.3%** ($2,433M) and Japan at **14.5%** ($1,291M).
- **Genre Polarization Across Geographies:**
  - **Western Markets (NA & EU):** Dominated by fast-paced and competitive categories—**Action** ($1,751M total global sales), **Sports** ($1,331M), and **Shooters** ($1,037M).
  - **Japan:** Demonstrates a distinct cultural divergence where **Role-Playing Games (RPGs)** represent the single largest share ($352M in JP sales), more than double the second-place Action genre ($160M).
- **Platform Lifecycles:** Home consoles from Sony (PS2, PS3), Microsoft (X360), and Nintendo (Wii, DS) drove historical peak industry volume. Nintendo and Electronic Arts stand out as the top global publishers by unit sales volume.

2. Community Engagement & Sentiment Patterns (games.csv)

- **Community Rating Stability:** Modern user review scores follow a concentrated distribution around **3.7 to 4.2 out of 5.0**, showing that players actively engage with titles they perceive favourably.
- **Intent vs. Consumption (Backlog to Wishlist Ratio):** High backlog-to-wishlist ratios highlight games that players purchased or intended to complete but postponed, representing high initial interest with delayed consumption.
- **Studio Reputation:** Dedicated developers (e.g., FromSoftware, Supergiant Games, Nintendo EPD) maintain strong average user scores (≥ 4.2), driving organic community retention through perceived quality rather than mass publishing volume alone.

3. Cross-Domain Strategic Correlations (clean_merged_vgsales)

- **Ratings vs. Global Sales Weak Correlation ($r \approx 0.07$):** High user review scores do **not** guarantee commercial blockbuster status. Mass-market titles often reach high sales volumes through brand IP recognition, marketing spend, and casual accessibility despite moderate reviews.
- **Wishlist vs. Rating Moderate Correlation ($r \approx 0.53$):** Community wishlists strongly align with game quality and critical reputation, serving as a reliable early indicator of player sentiment and prestige.
- **The "Engagement vs. Monetization" Gap:** Certain genres (notably Indie Adventure, RPGs, and Strategy) record elevated playthrough counts and backlogs relative to their historical unit retail prices, signalling monetization potential through DLC, live services, or digital re-releases.

Strategic Business Recommendations

Targeted Regional Portfolio Allocation:

Marketing and publishing teams should tailor releases geographically: prioritize Shooters and Sports in North America and Western Europe, while directing RPG and character-driven titles to Japan and Asian territories.

De-risking IP Development:

Because critical review scores alone do not dictate sales volume, publishers should track **early wishlist velocity** and **playthrough activity** rather than review aggregators alone when forecasting long-term demand and post-launch support budgets.

Monetizing the Backlog:

Titles with high backlog counts represent existing buyers who haven't finished the game. Quality-of-life patches, seasonal community challenges, and mid-tier DLC expansions can re-engage these dormant players and drive secondary monetization.